

t263 — mo:os public repo audit (moos-kernel, moos-router)

Audited from shallow clones (depth 50) of the two public Collider-Data-Systems repos, 2026-07-22 (T=263). Full agent reports in appendices; synthesis first.

Synthesis

The system. mo:os is a personal semantic OS: an append-only rewrite-log kernel (4 ops — ADD/LINK/MUTATE/UNLINK; `state = fold(log)`) with an operad type registry, URN-everything identity, WF01–WF21 relation families, provenance strata S0–S4, seat/occupancy gates (§M11/§M12), HDC-derived views, and twin sync. moos-router is the WF16 federation gateway fanning reads across per-workstation kernels (persona-named `hp-z440.{menno,lola,moos}`) and routing writes to a kernel-of-record by URN prefix. The private `ffs0` repo is the control workspace: ontology v3.16.2, topology SOT, PowerShell fleet launchers, and the fleet-level `AGENTS.md` (seat table + personas) that both repos' `CLAUDE.md/AGENTS.md` files declare as source of truth.

T-number decoded — from your own code. `internal/tday/tday.go`: integer day epoch since 2025-10-31 23:00 UTC. t263 = 2026-07-22. It's not a session counter; it's the fleet calendar. (Verified arithmetically; Nov 1 = day 0.)

Health verdicts.

- moos-kernel: strong. Tests 0.94:1 test:source LOC, all green incl. `-race`, zero panics, disciplined errors, exemplary decision docs. Gaps: no lint/gofmt gate (15 files currently unformatted), WF constant drift in `graph/relation.go` vs ontology, no benchmarks/fuzz, Windows code never CI-executed, `README↔CLAUDE.md` duplication drifting.
- moos-router: clean and small (1.0k source LOC, stdlib-only, race-clean) but **has no Go CI at all** — governance workflows exist, tests only run on dev machines. History carries 3 × ~9.6MB committed `.exe` blobs (12MB clone tax). No LICENSE despite README promise.

One real bug worth a work order. `handleFanout` is the catch-all for unmatched method+path and re-sends the original method — an unanticipated POST/DELETE to any path outside `/rewrites|/programs` broadcasts the write to every kernel, violating the "writes never fan out" invariant (proxy.go:171,332 vs README.md:29). Fix: restrict fan-out to GET / explicit allowlist + regression test. Adjacent: deprecated

`Router.{Rules,TypeRules,Peers,kernels}` mirror fields are written unsynchronized on reload (latent race, nothing reads them — delete).

Ranked next actions across both repos.

1. Router: add Go CI (build, vet, gofmt check, `test -race`) — ~15 lines of YAML, biggest gap.
2. Router: close the fan-out write-broadcast hole + delete dead mirror fields.
3. Kernel: add gofmt/golangci-lint gate to existing CI; format the 15 files.
4. Kernel: fix WF01–WF21 constant/label drift; add MOOS_INTEGRATION assertion constants↔ontology.
5. Kernel: `-race` + windows-latest CI legs (Windows lock code currently never tested in CI).
6. Kernel: fuzz URN parser / JSONL decoder / JSON-RPC; benchmarks for the 4 `TODO(perf)` sites.
7. Router: purge `.exe` blobs via git-filter-repo (needs ruleset lift + force-push — your call).
8. Router: `http.Server` with `ReadHeaderTimeout` + graceful shutdown; strip hop-by-hop headers; commit LICENSE.
9. Both: de-duplicate README↔CLAUDE.md operational tables (already diverged on `--auth-token-file`).

Plugin/skill conclusions from observed practice.

- Stack reality: Go stdlib-first (both repos), TS/React (Zappa, repo not yet located — likely private), Python (xDSL interest, ML), Postgres/Supabase, category theory as working vocabulary.
- Keep: engineering (code-review, testing-strategy, architecture, debug), design (Zappa UX), productivity, data, pdf-viewer, dataviz.
- Remove: BOTH zoom-plugin installs — no trace of Zoom development anywhere; frees ~30 skills of context noise. Marketing: no observed use; remove unless you market.
- Enterprise-search: keep only if the evening digest gets added.
- Build next: a `moos-dev` personal skill encoding the operad nomenclature bans (never "schema/edge/field/mutation/binding"), stdlib-first rule, `authored-by: <agent-urn>/<session-urn>` commit trailers, T-day stamp conventions, WF glossary — so every coding session speaks fleet. Your local `ffs0/dev/claude-skills` (13 `moos-*` skills) likely overlaps; reconciling them against this list is the first job once `ffs0` is readable.

Persona files found. Both repos carry CLAUDE.md (working context, nomenclature, safety) and AGENTS.md (thin mirrors pointing at `ffs0/AGENTS.md` as fleet SOT: seat table, fleet brief, personas). Your original "be the persona" instruction resolves there — sam-context v2 will fold in `ffs0/AGENTS.md` once the org grant lands. v1 corrections queued: T-day semantics, my-tiny-data-collider, mo:os glossary, GitHub handle MSD21091969.

Appendix A — moos-kernel full report

1. Purpose & architecture (5 lines)

Append-only rewrite-log kernel for a personal "semantic OS" (mo:os): the only state changes are 4 rewrite ops (ADD/LINK/MUTATE/UNLINK); `state(t) = fold(log[0..t])`, replay deterministic. Strict layer tower: `internal/graph` (pure types, no IO) → `fold` (pure catamorphism) → `operad` (ontology/type registry, read-only after load) → `kernel` (sole effect layer: Runtime, JSONL LogStore w/ single-writer lock, §M11 liveness + §M12 admin gates, governance sweep) → `transport/mcp` adapters (HTTP+SSE REST, MCP JSON-RPC over SSE/stdio/streamable-HTTP, optional QUIC/HTTP3). Side derivations: `internal/hdc` (hyperdimensional/spectral indexes recomputed per apply), `internal/reactive` (read-only predicate engine proposing envelopes), `internal/tday` (single time-epoch source). Single binary `cmd/moos` with flags, idempotent seed mode, graceful shutdown (SIGINT/SIGTERM, 5s timeout — `cmd/moos/main.go:246-258`). Ontology (v3.16.2, 53 node types, WF01–WF21) lives in a *sibling private repo* `ffs0`, loaded at boot — the repo self-describes as data-driven and "readback is truth over docs."

2. Stack facts

- Go 1.24 (go.mod); builds clean on go1.24.7 (29s cold), `go vet` clean. Module `moos/kernel` (non-canonical path, doesn't match GitHub URL — fine for a binary, blocks `go install`).
- Deps: genuinely stdlib-first. One direct dep, `quic-go v0.59.0` (gated behind `--quic-addr`); 5 transitive (qpack, x/crypto, x/net, x/sys, x/text). A `webtransport-go` spike was deliberately reverted at T=260 to restore the invariant (docs/spike-p9-webtransport.md documents the decision + re-adoption criteria — exemplary).
- Size: 75 .go files; 9,808 source LOC + 9,265 test LOC (~0.94:1 test:source). 10 packages. Single author (Sam Maassen, all 50 recent commits), master-only branch, active today.

3. Testing reality

- **Pass:** `go test ./...` all 9 packages OK in **4.1s**; `-race` on kernel+reactive also passes (2.5s). 293 test functions, 30 test files.
- Stdlib `testing` only (no testify/ginkgo — consistent with policy). Style is mostly bespoke scenario tests with rich intent comments (e.g. `internal/kernel/liveness_test.go` documents

why it bypasses validators); only 6 files use table-driven `range` loops, 2 use `t.Run` subtests, 9 `t.Helper` calls, **zero** `t.Parallel`.

- **Zero benchmarks, zero fuzz tests** — notable gap given a hand-rolled URN parser (graph/urn.go), JSONL log decoder, and JSON-RPC protocol layer, all classic fuzz targets; and TODO(perf) items with no benchmark to measure against.
- Coverage gaps: `cmd/moos` (0 tests), `internal/mcp` server/protocol (only `auth_test.go`), `transport/server.go`, `quic.go`, `twin.go` untested. Integration tests correctly env-gated (`MOOS_INTEGRATION=1`, `internal/operad/validate_color_gate_test.go:274`) since they need the sibling `ffs0` ontology file.

4. CI / tooling reality

- `.github/workflows/go-ci.yml`: build + vet + test on every PR and master push, `go-version-file: go.mod`, cache, minimal permissions — good but minimal, and only added recently (t260; before that "verified green" was a manual claim, per its own comment).
- `governance-sentinel.yml`: weekly cron verifying branch-protection ruleset still active, auto-files an issue on drift, fail-safe on token absence. Unusual and thoughtful.
- `project-sync.yml`: PR/issue → org Projects-v2 board sync, deliberately non-blocking.
- **Missing**: no Makefile/scripts (README bench script `scripts/bench-frame-read.mjs` referenced in docs doesn't exist in-repo), no `golangci-lint` or any lint config, **no gofmt gate — and 15 files currently fail `gofmt -l`** (cosmetic alignment drift, e.g. `internal/fold/errors.go`, `internal/graph/node.go`), no `-race` in CI, no coverage reporting, no Windows/macOS matrix despite platform-specific code (`kernel/log_lock_{unix,windows,other}.go` + a Windows-only migration test that CI never executes), no Dependabot/govulncheck, no releases/tags.

5. Practices observed

Good: exceptional doc-comment culture — comments carry provenance ("round-9 review finding — see PR #23", `tday.go`) and rationale, not restatement; disciplined error handling (16 sentinel errors in `fold/errors.go`, 84 `fmt.Errorf` with 35 `%w` wraps, `errors.Is/As` used, **zero panics** in non-test code); fail-closed defaults (unknown `FiringKind` rejected, color-gate §12.2); layering actually enforced by package structure (`graph` imports nothing impure); TODO discipline: only 7, all tagged `TODO(perf)/TODO(WF)` with `O()` analysis (`kernel/runtime.go:296,790`; `transport/twin.go:99,105`); decision records kept even for reverted work (spike doc); secrets policy explicit (CLAUDE.md "Safety"); commit messages: conventional-commit-ish `feat:/fix:/docs:/ci:/ops:` mixed with package scopes (`kernel:`, `operad:`, `fold:`) and T-day stamps (`T=248:`, `(t263)`), PR-merge workflow with adversarial (Copilot) review folded in — though two "Potential fix for pull request finding" commits are low-information. **Missing/drift**: doc/code divergence on WF vocabulary —

`internal/graph/relation.go:12-30` defines constants only WF01–WF19 with stale labels (WF13 "Promotion", WF19 "Session-local T counter") while README/registry/port_colors.go say WF13=governance-proposes, WF19=occupancy, WF20/WF21 exist only as data-driven strings; only 2 of 9 packages have `// Package x` doc comments (reactive, tday); auth defaults open when token unset (documented, warning-logged — acceptable for its threat model but worth a CI reminder); README duplicated ~60% into CLAUDE.md (drift risk, already showing: CLAUDE.md flag table has `--auth-token-file`, README's doesn't).

6. mo:os glossary

Term	Meaning	Where
Rewrite (ADD/LINK/MUTATE/UNLINK)	the only 4 state operations; log of rewrites is truth	README:12-22, graph/rewrite.go
fold / catamorphism	pure replay of log → state	internal/fold/evaluate.go, replay.go
operad	the type system/ontology registry (port pairs, validation) — "never say schema"	internal/operad/registry.go, CLAUDE.md nomenclature table
WF01–WF21	rewrite categories (relation families): WF01 owns, WF02 governs, WF03 hosts, WF13 governance-proposal, WF15 semantic (needs contract_urn), WF19 opens-on/has-occupant, WF20 grammar-promotion, WF21 causes (DAG-enforced)	graph/relation.go:12-30, README:168-171, operad/port_colors.go
URN	<code>urn:moos:<type>:<qualifier></code> , immutable identity (CI-3); observed schemes: user, program, session, rel, kernel, agent, t_hook, ki, claim, role, workstation, watcher, reactor	graph/urn.go:8-25

Term	Meaning	Where
Stratum S0–S4	provenance layers: Authored→Validated→Materialized→Evaluated→Projected (S4 never truth)	graph/stratum.go:5-17
Seat / occupancy	an agent "seated" on a session via WF19 has-occupant relation; basis of §M11 liveness gate	kernel/liveness_test.go:40, operad/occupancy.go
Workstation	physical host node (hp-laptop, z440), seeded WF01 owns / WF03 hosts	cmd/moos/main.go seed, README:115-119
Engine	reactive.Engine : watcher/reactor/guard evaluator, read-only, proposes envelopes	reactive/engine.go:15
Workspace	ffs0 = private control/research workspace (ontology SOT); §M18-§M20 "workspace shape" ports	README:176, operad/registry.go:73
T-day / t-number	integer day epoch since 2025-10-31 23:00 UTC; stamps commits/sessions (T=260)	internal/tday/tday.go:18-25
§M11/§M12/§M14	kernel spec sections: session-liveness gate, admin-capability gate, predicate kinds	kernel/liveness.go, reactive/predicate.go
t_hook / sweep	time-predicate hooks; 30s sweep proposes WF13 governance_proposals, never auto-applies	kernel/sweep.go, README:105-112
Twin / adjoint sync	M9 second-kernel mirroring via /twin/ingest	transport/twin.go

Term	Meaning	Where
HDC	hyperdimensional computing views (spectral, Fiedler, fiber, crosswalk) derived from state	internal/hdc/

7. AI-config / persona files

- **CLAUDE.md** (186 lines): full assistant working context — ontology pointer, enforced nomenclature table, gate semantics, package map, CLI flags, dev conventions, per-actor URN discipline for envelope authors, auth model, safety rules. Effectively the real contributor guide.
- **AGENTS.md** (35 lines): explicit "thin mirror" of `ffs0/AGENTS.md` (external SOT: seat table, fleet brief, personas live there, not in this repo); branching convention (`feat/<slug>`), `authored-by: <agent-urn>/<session-urn>` commit-trailer convention; footer trailer shows it was itself agent-authored (`agent:claudio-cowork.hp-z440`).
- No `.claudio/`, skills, or persona files in-repo; references to ffs0 skills/personas are pointers only.

8. Top 5 recommendations

1. **Add a formatting/lint gate to go-ci.yml** — `test -z "$(gofmt -l .)"` (or `gofmt-simplify`) + `golangci-lint` with a small config (`govet`, `staticcheck`, `errcheck`, `misspell`); fix the 15 currently unformatted files in one commit. Cheapest credibility win; CI already exists to carry it.
2. **Fix the WF vocabulary drift in `internal/graph/relation.go`** — add WF20/WF21 constants, correct the WF13/WF18/WF19 comment labels to match the ontology/README, and add a `MOOS_INTEGRATION` test asserting constants ↔ ontology registry agreement, since "nomenclature is enforced" is a stated core value.
3. **Run `-race` in CI and add a `windows-latest` matrix leg** — race passes locally today, so it's free; the Windows lock/migration code (`log_lock_windows.go`, `log_store_migration_windows_test.go`) is currently never executed by any CI.
4. **Add fuzz tests for the parsing surfaces** (`URN.Validate`, JSONL log decode in `log_store.go`, MCP JSON-RPC protocol.go) and benchmarks for the four `TOD0(perf)` hot paths (state Clone, gate loop, relation scans) so the perf TODOs become measurable.
5. **De-duplicate README ↔ CLAUDE.md** (flag table already diverged: `--auth-token-file` missing from README) — keep operational reference in one file and link; and either commit `scripts/bench-frame-read.mjs` or correct the spike doc's reference. Optionally add `govulncheck/Dependabot` for the quic-go family, the one non-stdlib exposure.

Appendix B — moos-router full report

1. Purpose & architecture (5 lines)

Stateless HTTP federation gateway (WF16) sitting in front of multiple `moos-kernel` instances at `:9000`. It fans read queries out to every known kernel and merges JSON results; routes writes (`POST /rewrites|/programs`) to exactly one kernel-of-record via type-map first, then longest-URN-prefix shard match; cascades `GET /state/nodes/{urn}` misses to peer routers on other workstations. Explicitly NOT a kernel: no rewrite log, no operad validation, no §M11/§M12 gates — "log-is-truth stays inside each moos-kernel" (README.md:7). Routing table comes from a shared topology JSON (`--topology-file`, in the private ffs0 repo), atomically hot-swappable via localhost-only `POST /admin/topology/reload`, with CLI `--shard/--default` flags as boot fallback. Layout: `cmd/router/main.go` (flags+serve) and `internal/proxy/` (proxy.go dispatch, topology.go table/reload).

2. Stack facts

- **Go 1.23** in go.mod; **zero dependencies** — go.mod is 3 lines, module `moos/router`. The "stdlib-first" claim is actually *stdlib-only*: verified, no go.sum exists. Builds fine on go1.24.7.
- **LOC**: 1,779 Go total — 1,035 source (main.go 138, proxy.go 672, topology.go 225) + 744 test.
- **Why 12MB**: working tree is ~120KB; `.git` is 12MB (pack 10.93 MiB) because **three ~9.6MB moos-router.exe Windows binaries were committed then untracked** (commits c68e606, 61c0950; untracked in 54183d8/8419feb/7767e1a "chore: untrack binary"). `.gitignore` (whole content: `moos-router.exe`) now blocks re-adds, but the blobs remain in history. No vendoring, no data files.

3. Testing reality

- **Framework**: stdlib `testing` + `httpptest` only. Not table-driven — 22 individual scenario functions (13 in `proxy_test.go`, 9 in `topology_test.go`), each spinning `httpptest` kernel stubs. Good use of `t.Helper()/t.TempDir()` (`topology_test.go:12`).
- **Results**: `go test ./... PASS` (pkg 0.069s, ~6.4s wall incl. compile); `go test -race ./... PASS` (1.098s pkg, ~34s wall). `go vet` clean, `gofmt -l` clean.
- **Coverage of behavior**: routing (longest-prefix, priority tiebreak, no-match), write-proxy body fidelity, fan-out merge/partial-success/all-fail, healthz incl. timeout, type-routing

precedence+fallback, reload swap/reject-empty/dangling-ref/tailscale-skip, localhost guards. Solid for its size.

- **Gaps:** `cmd/router` has **no tests** (flag parsing, boot fallback logic untested); **zero benchmarks; no integration tests** against a real moos-kernel (explicitly declared out of scope, AGENTS.md:34, but no cross-repo contract test exists either); peer-cascade path (`handleRoutedNodeRequest`) has no direct test.

4. CI/tooling reality

- **No Go CI at all.** The two workflows never compile or test the code: `governance-sentinel.yml` (weekly cron verifying branch-protection ruleset still active, files an issue on drift, fail-safe on missing PAT) and `project-sync.yml` (syncs PRs/issues to org Project v2 board #4, deliberately non-blocking). Both are well-commented and defensively written — but `go test` runs only on developer machines.
- No Makefile, no golangci-lint/staticcheck config, no Dockerfile, no systemd units, no release automation. Deployment = PowerShell launchers in the private ffs0 repo (`start_federation_{z440,laptop,hpprodesk}.ps1`, README.md:68). No LICENSE file despite README:102 promising "formal license" later.
- **Commit conventions** (36 commits total): early commits conventional-ish with tick scopes — `feat(T=157):`, `fix(T=158):`, `ci(T=159):`, `docs(readme):` — drifting later to plain sentences. Provenance trailer convention `authored-by: <agent> / <session> / <purpose-slug>` (AGENTS.md:31) applied on 5 recent commits, incl. one URN-form (`urn:moos:agent:vscode.hp-laptop.copilot`). PRs #1–#11 merged via GitHub; Copilot review feedback folded in explicitly. Minor noise: duplicated commit message pair (c038c87/1577e59), two author emails for the same person.

5. Practices observed

Good: atomic `atomic.Pointer[TopologyTable]` snapshot swap for lock-free hot reload (topology.go:74-94); fail-closed loopback guard using `RemoteAddr` only, never `Host` header (proxy.go:408-419); reload refuses empty/dangling tables so a bad file can't brick a live router (topology.go:153-174); fail-loud on missing `--local-host` to prevent Tailscale hairpin misrouting (main.go:114-119); per-upstream 5s/2s timeouts; consistent godoc comments; **zero TODO/FIXME/HACK markers**; exceptional README precision (per-route dispatch table with status-code semantics, README.md:24-31).

Issues found:

- `handleFanout` is the catch-all for every unmatched method+path and re-sends `req.Method` — so a **POST/DELETE to any path other than `/rewrites|/programs` is broadcast to ALL kernels** (proxy.go:171, 332), contradicting the "writes are never fanned out" invariant (README.md:29) for unanticipated endpoints.

- Deprecated mirror fields `Router.{Rules,TypeRules,Peers,kernels}` are written unsynchronized by `Reload()/SetPeers()` (proxy.go:52-57, 106-110) — latent data race if concurrent admin reloads occur; handlers don't read them, so they're pure liability.
- Plain `http.ListenAndServe` (main.go:135): no `ReadHeaderTimeout`, no graceful shutdown/signal handling.
- `copyHeaders` forwards hop-by-hop headers (Connection etc.) verbatim and sets no `X-Forwarded-For` (proxy.go:654-660).
- Boot log prints CLI flag counts even after the topology file replaced the table (main.go:132-133) — misleading operator signal.

6. mo:os glossary additions (file refs)

- **WF16** — cross-kernel federation read cascade + URN-routed write proxy; this repo (README.md:3, AGENTS.md:6). **WF01–WF21** — kernel-side `rewrite_category` workflow validator range (README.md:94; CLAUDE.md:70 says WF01–WF20, one-off drift).
- **Four rewrites** — ADD · LINK · MUTATE · UNLINK, the only mutations (AGENTS.md:17).
- **Log-is-truth / fold** — append-only rewrite log per kernel; state derived by folding (README.md:7). **HG folded state** — top of the SOT hierarchy (AGENTS.md:10).
- **\$M11 / \$M12** — kernel session-liveness and admin-capability gates (README.md:7).
- **Operad** — kernel type system; banned vocabulary: edge, wire, field, mutation, schema, association, binding, `_ref` (AGENTS.md:17).
- **URN prefixes** — `urn:moos:kernel:<name>`, `urn:moos:ws:<host>`, `urn:moos:session:*`, `urn:moos:channel:*`, `urn:moos:agent:*` (topology.go:167; proxy_test.go:401; git trailer).
- **Shard** — URN-prefix→kernel-URL rule; longest prefix wins, priority desc tiebreak (proxy.go:22-26). **Kernel-of-record** — the single write target (README.md:3). **Type-map** — `TypeID`→kernel "routing functor", checked before shards, survives reloads (README.md:40-42).
- **Workstations/seats** — hp-z440, hp-laptop, hp-prodesk; kernels persona-named `hp-z440.{menno,lola,moos}` (README.md:83-85). **Peers** — sibling routers per workstation (main.go:43). **http_local vs http_tailscale** — same-host vs remote kernel addressing; hairpin guard (topology.go:139-144).
- **T=nnn** — project tick stamps in commit scopes (T=157...166; "since T=172" CLAUDE.md:6; "T=247 takeover" project-sync.yml:21).
- **ffs0** — private control workspace/KB holding topology SOT + launchers (README.md:95). **moos-config** — LEGACY, do not use (README.md:96). **SOT** — source-of-truth hierarchy: HG state > ffs0/AGENTS.md > repo docs (AGENTS.md:8-14).

7. AI-config / persona files

- **CLAUDE.md** — Claude agent instructions: repo role, the "NOT a kernel" rule, flags, safety (never log envelope bodies, never commit secrets/binary).
- **AGENTS.md** — self-described "thin mirror" of CLAUDE.md for Copilot/Cursor/Codex/Gemini/Antigravity; adds SOT hierarchy, nomenclature bans, branching (**feat/<slug> off master**), provenance-trailer rule; points to fleet SOT **ffs0/AGENTS.md**.
- README.md:105 and both files carry **authored-by:**
agent:claude-cowork.hp-z440 / session:sam.z440-cowork-workspace /
 ... trailers — docs are agent-authored with provenance. No **.claude/**, **claude-skills**, or persona directories in-repo (personas live as kernel names, e.g. **hp-z440.menno/.lola**).

8. Top 5 recommendations (ranked)

1. **Add a Go CI workflow** (**go build**, **go vet**, **gofmt -l** check, **go test -race** **./...** on push/PR, pin toolchain). The repo has governance CI but nothing ever runs its own tests; this is the single largest practices gap and costs ~15 lines of YAML.
2. **Purge the three **moos-router.exe** blobs from history** with git-filter-repo — clone shrinks ~12MB → ~150KB. Requires a coordinated force-push (temporarily lift the Q2 ruleset the governance-sentinel watches, then re-apply).
3. **Restrict **handleFanout** to GET** (or an explicit method/path allowlist, 405 otherwise) so unmatched POSTs can't broadcast writes to every kernel; add a regression test asserting the "writes never fan out" invariant.
4. **Delete the deprecated **Router.Rules/TypeRules/Peers/kernels** mirror fields** (unsynchronized writes on reload = latent race; nothing reads them) and fix the boot log to report the live table, not CLI flag counts.
5. **Production-server hygiene + licensing**: swap **ListenAndServe** for an **http.Server** with **ReadHeaderTimeout** and signal-driven graceful shutdown; strip hop-by-hop headers in **copyHeaders**; commit the LICENSE file the README already promises. (Also worthwhile: a couple of **cmd/router** flag-parsing tests and a table-driven rewrite of the **Route()** tests.)